

Open Problems in Heterogeneous Distributed Inference

Hanzo AI — ML Systems

Research-agenda companion to the distributed-inference series

Abstract

Three companion papers report a working heterogeneous distributed-inference stack. One engine shards a model tensor-parallel across an AMD/NVIDIA boundary over commodity ethernet and decodes coherently [1]; an analytical model derives the latency physics of geo-distributed autoregressive decode and the speculative-decoding cure [2]; and a three-plane blueprint turns the trusted-LAN ring into a permissionless, consensus-managed, post-quantum-authenticated network [3]. Building them surfaced eight problems a homogeneous, single-trust, datacenter-local stack never has to confront, because each is an assumption that only *heterogeneity*, *wide-area distribution*, or *permissionlessness* puts under load. This paper states the eight precisely — with the formal object where one exists — and ties each to the companion that owns the relevant result, so the boundary between what is measured, what is designed, and what is open stays sharp. The eight are: (1) sampling-determinism boundaries in replicated-state inference; (2) quant-aware tensor-parallel sharding on block-quantized weights; (3) pipeline parallelism as the format-agnostic primitive and its layer-assignment ILP; (4) speculative decoding as a latency amortizer, made adaptive under round-trip time and acceptance drift; (5) the prefill launch-overhead wall, attacked by fixed-shape graph capture versus compiler-level fusion; (6) KV-cache wire compression for prefill migration and its cross-hop error accumulation; (7) the geo-distributed decode latency floor and the placement/replication optimization that follows from it; and (8) cheap verifiable inference for a permissionless compute network. None is a tuning parameter; each is a research question, and several compose — the layer schedule of (3) and the geo-placement of (7) are one joint optimization, and the determinism boundary of (1) and the cheap verification of (8) share the canonical-f32 substrate. We publish the agenda so the problems can be attacked on the same terms.

1 Introduction: where the assumptions break

A model-serving stack built for one datacenter, one vendor, and one operator gets to assume a great deal: every rank runs the same silicon and the same numeric format, the network between ranks is a nanosecond-scale NVLink fabric, and every node is trusted to compute what it claims. The distributed-inference series removes those assumptions one at a time — the fleet becomes vendor-mixed [1], the links become intercontinental [2], and the nodes become strangers who must be paid and verified [3] — and each removal exposes a problem the assumption had been hiding.

This paper is the ledger of what those removals expose. It is deliberately *not* a results paper: it states problems, not solutions, and its contribution is precision — each problem is pinned to the companion that owns the surrounding result, given its formal shape where it has one, and separated from the parts already solved so that no claim is duplicated across the series. We adopt the series’ discipline of labelling measured, designed, and open, and here everything is open by construction; the job is to say *exactly* what is open and why the environment forces it.

The eight problems fall into four families. **Correctness under replication** (§2): when several ranks must agree on a sampled token, where must their arithmetic be bit-identical? **The right primitive and its schedule** (§3–§4): tensor parallelism is correct but coupled to dequantized weights and to a per-layer collective; pipeline parallelism is format-agnostic but demands a hard scheduling decision. **Paying the wire** (§5–§8): the latency of a wide-area hop is on the critical path of every token, and the levers are speculation, compression, and placement. **Trust without an operator** (§9): a permissionless node can lie about its work, and full re-execution is too expensive to be the answer. We take them in that order.

2 Problem 1: determinism boundaries in replicated-state inference

Statement. In any inference topology where more than one rank holds replicated state that must stay coherent — most sharply, the tensor-parallel ring of [1], where each rank keeps its own KV cache and advances it token by token — every rank must sample the *same* next token, or the caches diverge and the streams silently fork. The cross-vendor ring makes the *reduced activations* agree bit-for-bit by exchanging them in a canonical **f32** wire and reducing with a canonical tie-break [1]. But the final logits are computed *after* the last reduction, locally, in each rank’s own 16-bit compute dtype (**f16** on the ROCm rank, **bf16** on the CUDA rank). Sampling is therefore *outside* the agreement the wire guarantees: at a near-argmax tie, **f16** and **bf16** rounding can select different tokens, and the ranks desync.

The formal object. Call the *agreement set* $\mathcal{A}$ the minimal set of tensors that must be bit-identical across replicas for state to stay coherent. The canonical-**f32** reduce places the post-attention and post-MLP activations in $\mathcal{A}$. Coherent decode additionally requires the *sampled token id* $id \in \mathcal{A}$; the open question is the cheapest way to put it there. Two mechanisms bound the design space:

- **Broadcast the decision.** One head rank samples and broadcasts the token id; the others do not sample at all. Cost: one small (≤ 8 -byte) broadcast per token on the critical path. Correct by construction for *any* sampler (greedy, temperature, top- p , min- p), because there is exactly one draw.
- **Canonicalize the computation.** Every rank computes logits in canonical **f32** with a deterministic tie-break and a shared RNG stream, so all ranks sample identically without communicating. Cost: an **f32** logit head on every rank and a synchronized RNG; fragile the moment any op on the path is not bit-reproducible across vendors.

The trade is broadcast latency versus redundant canonical compute, and it interacts with the sampler: stochastic sampling needs a *shared seed and a shared draw*, which effectively forces the broadcast mechanism, so the “canonicalize” route is really only clean for greedy decode.

What is open. A general theory of determinism boundaries for replicated-state inference: which ops must be canonical, the cost of canonicalization versus decision-broadcast as a function of vocabulary size and ring width, and the extension to speculative decoding (§5), where the boundary must hold not for one token but for a verified block of K . This is the correctness dual of the verification problem of §9: the same canonical-**f32** property that makes two honest ranks agree is what lets a third party *check* that they did.

3 Problem 2: quant-aware tensor-parallel sharding

Statement. Tensor parallelism shards each weight matrix along rows (column-parallel) or columns (row-parallel), giving each rank a contiguous slice of the matrix [6]. The block-quantized formats that make edge inference fit in commodity memory — GGUF k -quants and their kin — store weights in *superblocks*: a run of (typically) 256 weights sharing a block scale, with sub-block scales and a packed low-bit mantissa. A naive row/column shard cuts *through* a superblock, so a rank’s slice no longer contains the shared scale its weights are quantized against. The only way to shard such a layout with an arbitrary cut is to dequantize the whole matrix, slice, and requantize per shard — which reintroduces the dequant–requant loss the format exists to avoid, and defeats the memory saving by materializing the dense matrix. This is why the shipping TP path in [1] is wired only through the dequantized (safetensors) model definitions and cannot shard a GGUF model at all.

What is open. *Quant-aware sharding*: a split that respects superblock boundaries. If the shard count divides the matrix dimension into superblock-aligned pieces (or the matrix is padded to make it so), each rank receives whole superblocks with their scales intact and no dequantization is needed. The open questions are the constraints and their cost: which (hidden size, head count, shard count, block size) combinations admit an aligned split; how much padding waste an arbitrary combination incurs; whether a format with per-shard scale replication can be sharded losslessly at any cut; and whether the analysis differs for row- versus column-parallel layers, which cut different axes of the superblock grid. The prize is TP over quantized weights — but note that pipeline parallelism (§4) sidesteps the entire problem, because it ships *activations*, not weight slices, across the seam and each node loads its layer range in whatever quantization it likes. Problem 2 is worth solving only where TP is worth keeping: inside a node or an NVLink domain, for a matrix too wide to hold on one accelerator.

4 Problem 3: pipeline parallelism as the universal primitive, and its layer-assignment ILP

The engineering result (in progress). The heterogeneous-cluster analysis of [1] argues, and the latency analysis of [2] proves under a wide-area cost model, that *pipeline parallelism* (PP) is the right primitive for a heterogeneous fleet: it is format-agnostic (§3), carries only an activation tensor across each node boundary rather than an all-reduce per layer, imposes no head-count-divides-node-count constraint, and reduces the sampling determinism boundary (§2) to a single head sampler. Implementing PP as the default multi-node primitive in the same engine is an engineering task under way in the codebase; it is not the open problem. The open problem is what PP forces you to decide that TP does not.

The open problem: optimal layer-range assignment. PP must cut the L -layer model into N contiguous ranges and assign each to a node, under nodes that differ in *compute*, *memory*, and *link bandwidth* simultaneously. Let node j have compute throughput ρ_j (layers/s-equivalent) and memory capacity M_j ; let layer l have compute weight w_l and resident footprint m_l (weights plus its KV cache). A pipeline is a permutation π of nodes onto stages and a contiguous partition $0 = c_0 < c_1 < \dots < c_N = L$ assigning layers $[c_{j-1}, c_j)$ to stage j . Steady-state pipelined throughput

is set by the slowest stage, so the assignment problem is

$$\min_{\pi, c} \max_{1 \leq j \leq N} \max \left(\underbrace{\frac{1}{\rho_{\pi(j)}} \sum_{l=c_{j-1}}^{c_j-1} w_l}_{\text{stage compute}}, \underbrace{\frac{B_{\text{hop}}}{b_{\pi(j), \pi(j+1)}}}_{\text{activation transmit}} \right) \quad \text{s.t.} \quad \sum_{l=c_{j-1}}^{c_j-1} m_l \leq M_{\pi(j)} \quad \forall j, \quad (1)$$

where B_{hop} is the per-hop activation size and $b_{\cdot, \cdot}$ the link bandwidth between consecutive stages. For a *fixed* node ordering π the inner problem — partition a weighted sequence into N contiguous parts minimizing the bottleneck part under per-part capacity — is solvable exactly by dynamic programming or parametric search. The difficulty is the *joint* choice of ordering π (which node is adjacent to which, hence which link carries each hop) together with the partition and, in the geo setting (§8), the placement of nodes in space: that couples an assignment, a sequencing, and a facility-location decision, and is NP-hard in general.

What is open. A practical solver: good heuristics with a bounded approximation ratio for Eq. (1); online *re-balancing* as nodes join, leave, or thermally throttle (the membership of a permissionless network [3] is not static); and the memory-vs-compute tension when the throughput-optimal cut violates a small node’s capacity. The single-stream *latency* objective — $\sum_j$ stage compute + T_{ring} rather than the bottleneck — is the variant [2] analyses; Eq. (1) is its throughput dual, and a real scheduler must weigh the two. Prior PP scheduling work assumes homogeneous stages [7] or homogeneous datacenter nodes [8]; the open case is genuinely heterogeneous ρ_j , M_j , and b_{jk} at once.

5 Problem 4: speculative decoding as a geo-distributed latency amortizer, made adaptive

What the companion owns. The latency paper [2] derives the speculative-decoding cure in full: a local draft proposes K tokens, the distributed target verifies all K in one pipeline pass, and at acceptance rate α the effective rate is $r_{\text{spec}}(\alpha, K) = \frac{1-\alpha^{K+1}}{1-\alpha} / (Kc_d + C + T_{\text{ring}})$, which recovers interactive speeds on an intercontinental ring by paying the round-trip once per accepted *block* rather than once per token. That derivation, its optimal K^* for fixed α , and the numbers are the companion’s result and are not restated here.

What is open. The companion treats α as a fixed parameter and K as a tuned constant. In deployment neither holds. The acceptance rate α *drifts* within a single generation — it is high on boilerplate and low at genuine decision points, and it depends on prompt, temperature, and how far the draft and target have diverged in context — so the throughput-optimal K is a moving target, not a launch-time flag. The open problem is an *online* controller that adapts K to the measured acceptance and the measured ring latency T_{ring} : estimate α from a trailing window of accept/reject outcomes, react to T_{ring} jitter and packet loss that the constant- ℓ model omits, and set K to the running optimum with a stability guarantee (a controller that chases a noisy α can oscillate). Two couplings make it harder and are themselves open: the determinism boundary of §2 must hold across the *whole verified block*, not one token, so a replicated target must agree on the accept/reject cut; and the draft can itself be placed on a nearby, more reliable node [3], making draft placement part of the scheduling problem of §4. The genuinely empirical unknown —

measured α for real draft/target pairs on real prompt distributions, which the companion explicitly does not claim — gates any adaptive scheme.

6 Problem 5: the prefill launch-overhead wall

Statement. On identical mathematics, hanzo-engine prefill reaches a lower GPU utilization than `llama.cpp` not because any kernel is slow but because there are too many kernels. Profiled on an NVIDIA GB10 (node *spark*) running a 35B-class mixture-of-experts prefill, the engine sustains $\approx 79.3\%$ GPU utilization across $\approx 33,000$ kernel launches per step, against `llama.cpp`’s $\approx 97.3\%$ across $\approx 3,000$ — an order of magnitude more launches for the same work. Per-kernel *busy* time is already at parity; the gap is entirely the idle interstice between launches, i.e. launch and scheduling overhead, not arithmetic. Decode does not have this problem, because the decode path is captured into a CUDA graph that replays the whole token as one submission; the prefill path is not, because mixture-of-experts routing is *data-dependent* — which experts fire depends on the tokens — so the shape of the work changes per step and a naive static graph capture does not apply.

Two attacks, and they compose. There are two structurally different ways to collapse the launch count, and they are not exclusive:

- **Fixed-shape chunked graph capture.** Quantize the data-dependent dimension: route to a fixed capacity per expert (pad or drop to a chunk size), so the per-step work has a bounded set of shapes, and capture a CUDA graph per shape. This keeps the many small kernels but pays their launch cost *once* at capture and replays them as one submission, exactly as decode already does. It is a runtime technique and inherits the graph machinery the engine has.
- **Compiler-level DAG fusion.** Reduce the *number* of kernels rather than the cost of launching them: fuse elementwise and reduction chains into fewer, fatter kernels at compile time, so there is less to launch in the first place. This is the subject of the kernel-fusion companion [5], where fusion is shown to be composition; extending it from local elementwise chains to the full prefill DAG (across the MoE gate, the expert GEMMs, and the combine) is the open compiler problem.

Graph capture attacks the *launch cost*; fusion attacks the *launch count*; applied together they multiply. The decisive reason to invest in fusion as well as capture is portability: graph capture is a CUDA (and partial ROCm) facility, but fusion is a property of the kernel *source* and therefore transfers to every backend the engine targets — Metal, Vulkan, and ROCm, where graph support is weaker or absent [4]. The open problem is the fusion *partitioner*: given the prefill DAG, choose the kernel boundaries that minimize launches subject to register and shared-memory limits, and do it once, portably, from the one source.

7 Problem 6: KV-cache wire compression for prefill migration

Statement. In steady-state pipeline decode the KV cache never crosses the wire — each node keeps the keys and values for its own layers [2]. The exception is *migration*: moving a prefilled context between nodes, as in disaggregated prefill/decode, session hand-off, or re-placement when the schedule of §4 changes. Then the KV cache — the large, context-growing state — must be shipped, and its size, not the activation’s, sets the transfer cost. The engine already has an FP8 KV representation, so the obvious move is to ship KV in FP8 and halve the bytes.

What is open. Error accumulation across hops. A single FP8 round-trip of the KV cache is a tolerable, well-studied quantization. The open question is what happens when compression is applied *repeatedly* — a context migrated across several nodes over its lifetime, or a design that keeps KV in FP8 resident and re-quantizes on each write — so that quantization error compounds hop over hop and layer over layer. Formally: bound the accumulated error of H successive FP8 (de)compressions of a KV cache as a function of H , context length, and layer depth, and identify the schemes that arrest it — error feedback (carry the residual forward), selective precision (keep the attention-sink and recent tokens in higher precision), or a migration protocol that recompresses from a retained high-precision anchor rather than from the last lossy copy. This is the compression dual of the determinism boundary (§2): the all-reduce must be canonical `f32` because it is summed every layer, but the KV cache may tolerate FP8 precisely because it is *read*, not reduced — and the open problem is exactly how much, and for how many hops.

8 Problem 7: the geo-distributed decode latency floor and the placement it dictates

What the companion owns. The latency paper [2] establishes the floor: single-stream autoregressive decode must traverse every pipeline stage per token, so its rate is $1/(C + T_{\text{ring}})$ where T_{ring} is the one-way loop latency, and because T_{ring} is bounded below by the speed of light in fiber ($\approx 4.9 \mu\text{s}/\text{km}$), an intercontinental ring floors decode at a few tok/s *regardless of accelerator speed*. The companion also states the levers as placement laws — regional-first placement, speculate across every slow boundary, co-locate the ring turn, keep KV node-local — and proves pipeline parallelism dominates tensor parallelism over any slow link by $\sim 4L/N$. The floor, the laws, and the $4L/N$ result are the companion’s and are not restated.

What is open. The companion’s laws are heuristics, stated and argued but not optimized or measured end-to-end. Two things remain. First, *empirical validation*: the companion is explicit that it reports no wall-clock from a running geo deployment; a real multi-continent ring, with real jitter and loss, must confirm the $1/(C + T_{\text{ring}})$ floor and the speculative recovery. Second, and deeper, the *placement and replication optimization* the floor implies. If one ring cannot serve a global user base at interactive latency, the answer is *regional replicas* — multiple rings, each latency-local to a set of users, with requests routed to the nearest — and choosing how many rings, in which regions, holding which shards, to minimize the population’s tail latency under a memory and cost budget is a facility-location problem entangled with the layer-assignment ILP of §4: where a node sits in space changes both which users it is near and which link it contributes to its ring. Formalizing that joint placement–partition problem, and the routing that assigns each request to a ring, is open, and it is the geo-scale version of Problem 3.

9 Problem 8: cheap verifiable inference for a permissionless compute network

What the companion owns. The three-plane blueprint [3] builds the transport and control layers of a permissionless inference network: a post-quantum-authenticated ZAP wire that proves *who* sent a tensor, a Quasar consensus control plane that admits staked miners and agrees the placement map, and a Photon committee that can threshold-sign an (request, output) pair. For result integrity it proposes three composable mechanisms — threshold attestation of outputs, redundant

re-execution with the canonical-f32 property as a bit-exact cross-vendor oracle, and luminance reputation as the economic backstop — and labels them, honestly, as design. Authentication and committee formation are solved there and not reopened here.

What is open: verification that is cheaper than the work. Authentication proves who sent a tensor, not that the tensor is *correct*, and the blueprint’s strongest integrity mechanism — redundant re-execution — costs a full second computation on every checked request. That is fine as a sampled spot check but cannot be the common path: a network whose verification costs as much as its inference has no margin to pay honest miners. The open problem is *sub-linear* verification of correct inference. Candidate directions, each unproven for transformer decode:

- **Sampling-based logit audits.** A verifier recomputes a *random subset* of the claimed logits (a few positions, a few layers) and checks them against the miner’s committed output, catching a cheating miner with high probability at a fraction of the compute. The open question is the soundness — how large a subset defeats an adversary who corrupts only the tokens a user cares about — and how it composes with the canonical-f32 determinism (§2) that makes a recomputed logit comparable bit-for-bit in the first place.
- **Redundant-execution economics.** Treat verification as a game: honest computation must be the profit-maximizing strategy under a spot-check probability p , a stake bond, and a slashing penalty. The open problem is to price p , the bond, and the reward so that the *expected* cost of cheating exceeds its gain for a rational miner, while keeping the honest miner’s expected verification tax small — and to do so against collusion among a committee and grinding on the verification lottery, threatens the blueprint names but does not price.

The unifying observation is that the canonical-f32 wire built for cross-vendor *correctness* (§2, [1]) is the same substrate that makes inference *checkable*: two honest nodes agree bit-for-bit, so any disagreement is a proof of fault rather than a rounding artifact. Turning that from a full-re-execution check into a sampled, economically-sound one is the last open problem, and the one that decides whether a permissionless inference network can exist at all.

10 Synthesis

The eight problems are not independent, and Table 1 states their dependencies as plainly as their statements. Three throughlines run across them.

Canonical f32 is load-bearing twice. The same property that makes an f16 rank and a bf16 rank agree on a reduced activation (§2) is what lets a third party check a miner’s work without a rounding dispute (§9). Correctness under replication and verifiable correctness are the same substrate seen from two sides.

Scheduling is one problem at two scales. The layer-assignment ILP of §4 and the regional-replica placement of §8 are the LAN and geo faces of a single partition-and-place optimization; solving one in isolation leaves the other’s coupling on the table, and a real deployment sets both at once, with speculative-draft placement (§5) folded in.

Pipeline parallelism dissolves more than it creates. It sidesteps quant-aware sharding (§3), collapses the determinism boundary to one sampler (§2), and makes the graph-capture asymmetry a local choice (§6) — but it buys those with the scheduling problem of §4 and the migration cost of §7. The net is favourable, which is why the series argues for it; the residue is real, which is why it is here.

#	Problem	Exposed by	What is open (owning companion)
1	Sampling-determinism boundary	heterogeneity	minimal agreement set; broadcast vs. canonicalize cost [1]
2	Quant-aware TP sharding	heterogeneity	superblock-aligned splits without dequant-requant [1]
3	PP layer-assignment ILP	heterogeneity	Eq. (1): heuristics, online rebalance [1, 2]
4	Adaptive speculation	wide area	online K under drifting α , jittering T_{ring} [2]
5	Prefill launch-overhead wall	(single-node)	fusion partitioner; portable, composes with capture [5, 4]
6	KV wire compression	wide area	cross-hop FP8 error accumulation bound [2]
7	Geo decode latency floor	wide area	empirical floor; regional-replica placement [2]
8	Cheap verifiable inference	permissionless	sub-linear logit audits; slashing economics [3]

Table 1: The eight open problems, the environment property that exposes each, and the open kernel with the companion that owns the surrounding result. Problems 3 and 7 are one optimization at two scales; problems 1 and 8 share the canonical-f32 substrate. Problem 5 is exposed already single-node but is sharpest when prefill is distributed.

Finally, the register. Some of these are engineering with a known shape — adaptive K (§5), fixed-shape graph capture (§6), FP8 migration (§7) — and will yield to careful implementation and measurement. Others are genuinely research — the determinism-boundary theory (§2), the heterogeneous scheduling ILP (§4), sub-linear verification (§9) — and may not have clean answers. We have tried to say which is which, because the most useful thing an open-problems paper can do is not overclaim the difficulty of the easy ones or underclaim the depth of the hard ones.

Acknowledgement of scope

This paper contributes problem statements, not solutions, and every result it leans on belongs to a cited companion: the cross-vendor decode measurement and its residuals [1], the latency floor and the speculative cure [2], and the three-plane network with its verification design [3]. Where a formal object appears — the agreement set (§2), the assignment ILP Eq. (1), the error-accumulation bound (§7) — it is stated to make the open question precise, not claimed as solved. The measured numbers quoted (the launch-overhead profile of §6) are attributed to the hardware that produced them (NVIDIA GB10) and to the companion that carries the surrounding stack [4].

References

- [1] Hanzo AI — ML Systems. *One Cluster, Every Vendor: Cross-Vendor Tensor-Parallel LLM Inference over Commodity Ethernet*. Distributed-inference companion (measured).
- [2] Hanzo AI — ML Systems. *The Latency Physics of Geo-Distributed Autoregressive Decode*. Distributed-inference companion (analytical).

- [3] Hanzo AI — ML Systems. *The Three-Plane Compute Fabric: Consensus-Managed, ZAP-Transported Decentralized AI Inference*. Distributed-inference companion (architecture).
- [4] Hanzo AI. *Edge LLM Inference Across Commodity Accelerators: hanzo-engine at llama.cpp Decode Parity on Every Backend*. Paper 05 (measured).
- [5] Hanzo AI. *Fusion Is Composition: Kernel Fusion as the Functor Law*. Paper 07.1.
- [6] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, B. Catanzaro. *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism*. arXiv:1909.08053, 2019.
- [7] Y. Huang et al. *GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism*. NeurIPS, 2019.
- [8] D. Narayanan, A. Harlap, A. Phanishayee, et al. *PipeDream: Generalized Pipeline Parallelism for DNN Training*. SOSP, 2019.
- [9] A. Borzunov et al. *Petals: Collaborative Inference and Fine-tuning of Large Models*. arXiv:2209.01188, 2022.
- [10] Y. Leviathan, M. Kalman, Y. Matias. *Fast Inference from Transformers via Speculative Decoding*. ICML, 2023.
- [11] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, J. Jumper. *Accelerating Large Language Model Decoding with Speculative Sampling*. arXiv:2302.01318, 2023.